Licenciaturas em Engenharia Informática
Programação Web
Ano Lectivo de 2026/2027 – 1º Semestre
AULAS LABORATORIAIS
FICHA 4

APP DE GESTÃO DE LOJA - CONTINUAÇÃO

Nesta ficha laboratorial, continuaremos a desenvolver a aplicação exemplo Gestão de Loja uma das aplicações base da infraestrutura de aplicações de suporte que irão ser utilizadas no trabalho prático a realizar nas aulas laboratoriais.

Desse modo, execute as seguintes etapas:

3. Criação da Base de Dados da aplicação Gestão de Loja

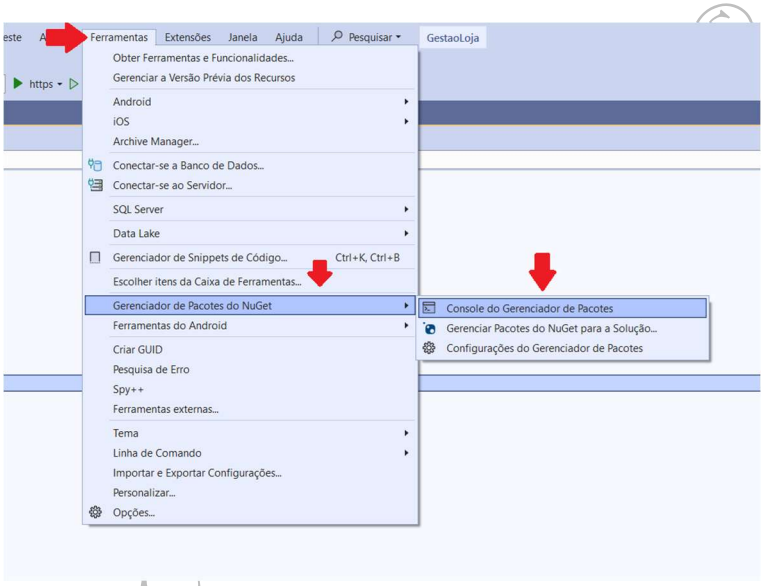


Figura 1- Opção do Menú "Console do Gerenciador de Pacotes"

Iremos agora criar a Base de Dados necessária associada à aplicação. Para tal, utilizaremos as instruções *add-migration **XYZ*** e *update-database* do conjunto de instruções designadas por *migrations*. Este conjunto de instruções *migrations*, contém mais instruções, mas essencialmente utilizaremos estas duas. Para isso execute os seguintes passos:

a. Como mostrado na Figura 1, abra o Console do Gerenciador de Pacotes, Figura 2, e digite o comando *Add-Migration BDInicial* (este comando não é case-sensitive e o nome, neste caso *BDInicial*, pode ser qualquer um mas logicamente devemos dar um nome a cada *migration* que seja descritivo do que se pretende fazer.

PWEB ISEC ©JB2024

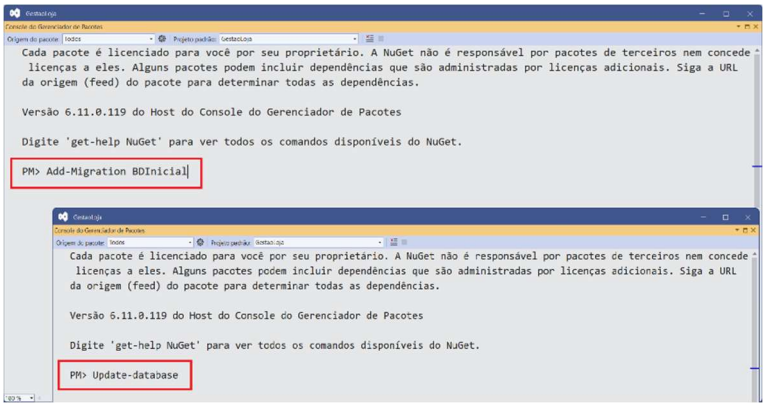


Figura 2- Console do Gerenciador de Pacotes – Add_Migration + UpdateDatabase

b. De seguida, nesta mesma consola, execute a instrução migration *update-database*. Após esta segunda instrução, se não ocorreram erros, a base de dados foi criada.

Esta base de dados foi criada com as tabelas cujas classes *C#* foram indicadas na classe *ApplicationDbContext*. Cada uma das tabelas corresponde a cada uma das classes identificadas que por sua vez correspondem às entidades identificadas no ***MER*** da base da dados da aplicação. Os campos das tabelas criadas correspondem às propriedades das classes de domínio que por sua vez correspondem aos atributos de cada entidade identificada no ***MER***. As classes de domínio C#, para além das instruções C# relativas às propriedades, "atributos", contêm instruções C# que corresponderão aos relacionamentos entre entidades.

Assim, partindo do *MER*, chegamos às classes C# de domínio e depois, aplicando as instruções *migrations*, obtemos as tabelas na base de dados, isto é, a base de dados.

Isto é possível porque o *Entity Framework* é um ***ORM***, *Object-relational mapping* o que nos permite trabalhar só com objectos, neste caso as classes C# correspondentes às entidades, e não precisamos de criar "manualmente" a base de dados recorrendo por exemplo a uma interface gráfica do *SGBD* que iremos utilizar.

Deste modo, para termos uma base de dados "bem-feita", temos de ter cuidado na criação das classes de domínio e das respectivas relações entre as classes.

c. De seguida, verificamos se a base de dados foi criada e se está de acordo com o pressuposto, isto é com o MER que definimos analisada a aplicação a desenvolver, os seus casos e de uso e as restrições e funcionalidades respectivas. Para isso, e como mostrado na Figura 3, acedemos à base de dados.

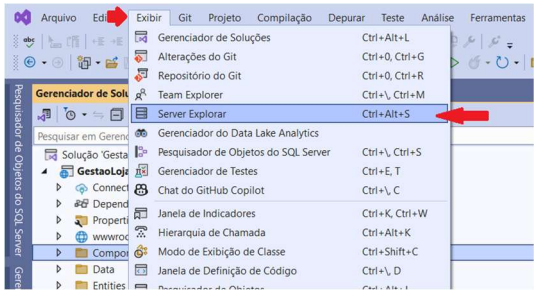


Figura 3- Opção do menú para aceder ao Server Explorer, servidor das BD

PWeb ISEC Coimbra 2026/2027 ©JB2024

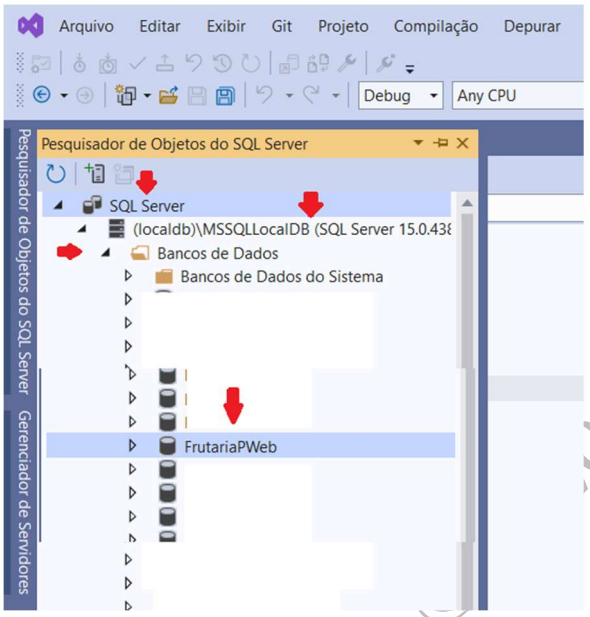


Figura 4- Base de Dados *FrutariaPWeb* criada no SQL Server LocalDB

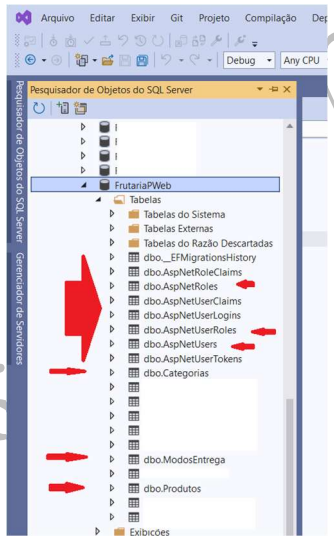
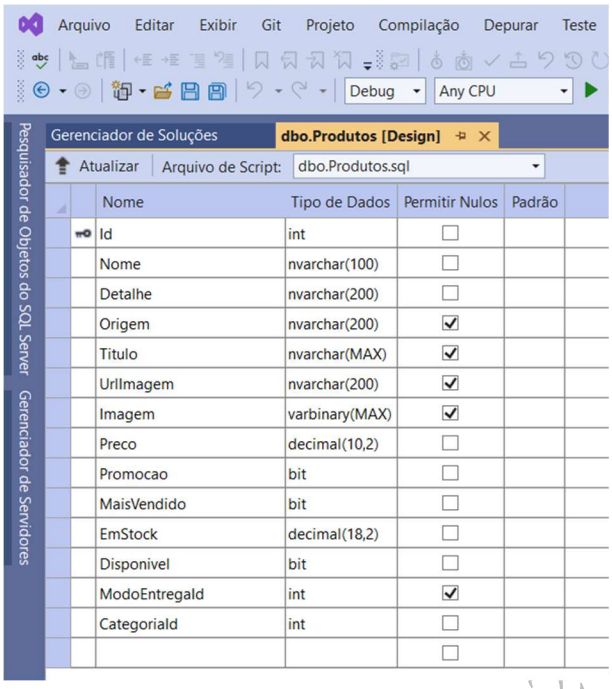


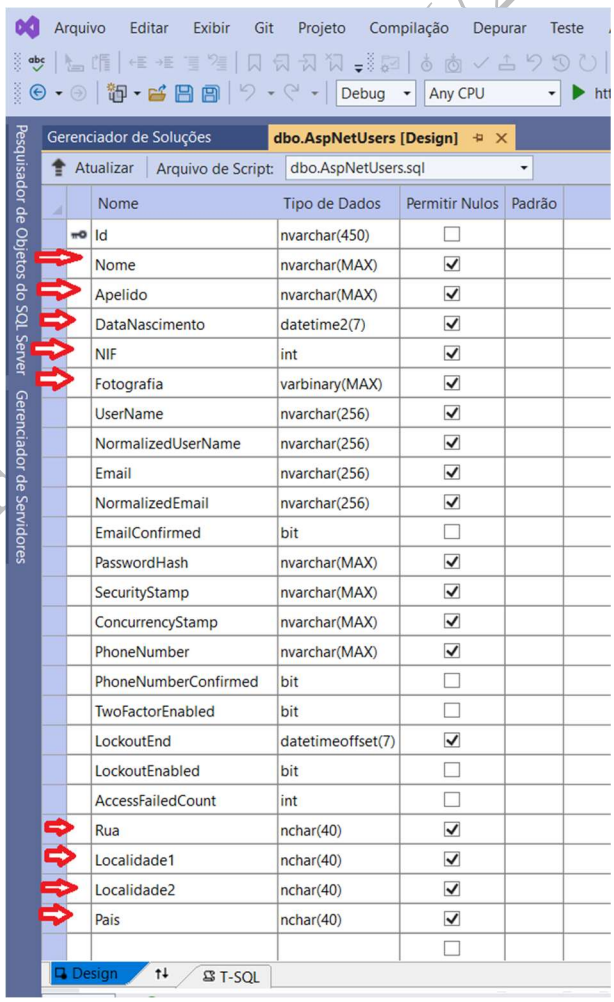
Figura 5- Tabelas da Base de Dados *FrutariaPWeb*

PWeb ISEC Co 2027 © 024



Nome	Tipo de Dados	Permitir Nulos	Padrão
Id	int	☐	
Nome	nvarchar(100)	☐	
Detalhe	nvarchar(200)	☐	
Origem	nvarchar(200)	☑	
Titulo	nvarchar(MAX)	☑	
UrlImagem	nvarchar(200)	☑	
Imagem	varbinary(MAX)	☑	
Preco	decimal(10,2)	☐	
Promocao	bit	☐	
MaisVendido	bit	☐	
EmStock	decimal(18,2)	☐	
Disponivel	bit	☐	
ModoEntregaId	int	☑	
CategoriaId	int	☐	
		☐	

Figura 6- Detalhe da tabela Produtos na Base de Dados *FrutariaPWeb*



Nome	Tipo de Dados	Permitir Nulos	Padrão
Id	nvarchar(450)	☐	
Nome	nvarchar(MAX)	☑	
Apelido	nvarchar(MAX)	☑	
DataNascimento	datetime2(7)	☑	
NIF	int	☑	
Fotografia	varbinary(MAX)	☑	
UserName	nvarchar(256)	☑	
NormalizedUserName	nvarchar(256)	☑	
Email	nvarchar(256)	☑	
NormalizedEmail	nvarchar(256)	☑	
EmailConfirmed	bit	☐	
PasswordHash	nvarchar(MAX)	☑	
SecurityStamp	nvarchar(MAX)	☑	
ConcurrencyStamp	nvarchar(MAX)	☑	
PhoneNumber	nvarchar(MAX)	☑	
PhoneNumberConfirmed	bit	☐	
TwoFactorEnabled	bit	☐	
LockoutEnd	datetimeoffset(7)	☑	
LockoutEnabled	bit	☐	
AccessFailedCount	int	☐	
Rua	nchar(40)	☑	
Localidade1	nchar(40)	☑	
Localidade2	nchar(40)	☑	
Pais	nchar(40)	☑	
		☐	

Figura 7- Detalhe da tabela Users na Base de Dados *FrutariaPWeb*

PWeb ISEC ©JB2024

d. Como se constata nas figuras anteriores, em particular na Figura 5, criamos uma base de dados chamada *FrutariaPWeb* cujas tabelas, campos e relações correspondem às classes C# de domínio que definimos no código da aplicação.
Observe-se que para além das tabelas Categorias, Produtos e ModosEntrega, correspondentes às classes de domínio C#, também foi criado um conjunto de tabelas relativas à gestão de utilizadores, em particular as tabelas *AspNetUsers*, *AspNetRoles* e *AspNetUsersRoles*. Estas tabelas foram criadas porque definimos no projecto a utilização de "Contas Individuais" o que levou, como já referido antes, a que na classe *ApplicationDbContext* em lugar de considerar como classe base a classe *DbContext* (classe base a utilizar caso não se use no projecto o Identity Framework), utilizou-se uma extensão, derivação, dessa classe a classe base *IdentityDbContext*. Estas extensões permitiram então criar estas tabelas relacionadas com os utilizadores. A utilização da classe de extensão das propriedades dos utilizadores, a classe *ApplicationUser* permite que na tabela *AspNetUsers* tenham sido incluídas os campos correspondentes às propriedades da classe *ApplicationUser*, como salientado na Figura 7, e isto sem ser preciso criar classes *C#* especificas para os *Users*, *Roles* e para a relação *Users x Roles*.

Caso a base de dados que criou seja igual à mostrada nas figuras atrás referidas, pode avançar para a parte seguinte; caso contrário tem de corrigir o código de modo a que a base de dados de modo corresponda ao mostrado,

PWeb ISEC Coimbra 2026/2027 © JB2024